CIS 613: System Testing

1

System Testing

- "Intuitively clear"
  - customer expectations
  - close to customer acceptance testing
- BUT we need a better basis for really understanding system testing
- Threads—the subject of system testing
- How are they identified?
  - ad hoc?
  - from experience?
  - from a possibly incomplete requirements specification?
  - from an executable model? (Model-Based Testing)

2

Threads of Execution

- An execution time concept
- Per the definition, a thread can be understood as a sequence of atomic system functions.
- When a system test case executes
  - a thread occurs, and
  - can be observed at the port boundary of the system
- The BIG Question: where do we find (or how do we identify) threads?
- Our approach—Model-Based Testing

3

Views of Threads

- A scenario of normal usage
- A use case or user story
- A stimulus/response pair
- Behavior that results from a sequence of system-level inputs
- An interleaved sequence of port input and output events
- A sequence of transitions in a state machine description of the system
- An interleaved sequence of object messages and method executions
- A sequence of machine instructions
- A sequence of source instructions
- A sequence of MM-paths
- A sequence of atomic system functions (to be defined)

4

Sources of Threads

- A “Expanded Essential Use Case”
  - pre-conditions
  - interleaved sequence of input and output events
  - post-conditions
- A path in an executable model
  - Finite State Machine
  - Event-Driven Petri Net

We are going to look at paths in a FSM

5

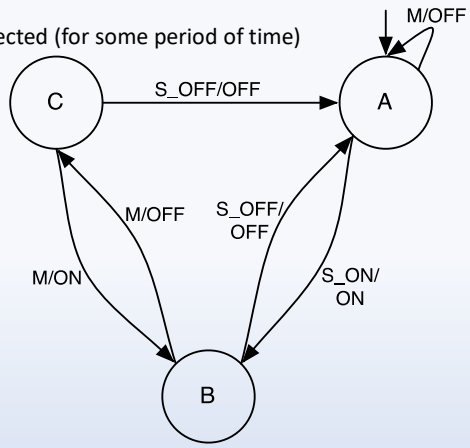
What are the threads here?

Input events: (M, S_ON, S_OFF)

- M = Motion Detected / Not Detected (for some period of time)
- S_ON = Switch Turned On
- S_OFF = Switch Turned Off

Output events (ON, OFF):

- Lights Off
- Lights On



6

Always the Case

When there is not Enough Time for Testing

- Operational Profiles
  - Need “traffic” estimates
  - Focus testing on the most likely modes of system usage
- Risk-Based Testing
  - An extension to Operational Profiles
  - Still need “traffic” estimates
  - Also need cost penalties for failed transactions
  - Focus on testing “the most potentially expensive” failures

7

Operational Profiles

- Zipf’s Law: 80% of the activities occur in 20% of the space
  - productions of a language syntax
  - natural language vocabulary
  - menu options of a commercial software package
  - area of an office desktop
  - floating point divide on the Pentium chip
- Rationale for operational profile testing
  - a small fraction of all possible threads represents the majority of system execution time
  - find the occurrence probabilities of threads and use these to order thread testing.

8

Operational Profiles

- A shortcut when test time is limited
- Estimate transition probabilities in a high level finite state machine
- Product of probabilities on a path is the probability of that path executing
- Method
  - Estimate probabilities
  - Compute path probabilities
  - Sort paths most to least likely
  - Test most likely paths until time expires

9

FSA Path Probabilities

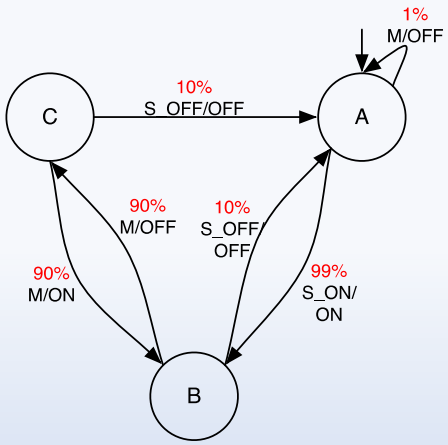
Automatic Light Switch Finite State Machine

Thread (State, Action):

- A, Turn switch on (S_ON)
- B, Leave room (M)
- C, Enter room (M)
- B.

Probability (State, Action = %):

- A, S_ON: 99%
- B, M: 90%
- C, M: 90%
- B = 99%*90%*90% = ~80%



10

In Class Assignment: Identify Thread Chances

Automatic Light Switch Finite State Machine

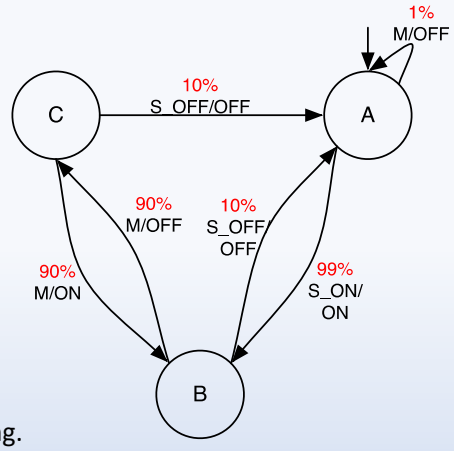
Identify 3 threads:

1. One should be *likely*
2. One should be *unlikely*
3. One should be *between*

For each thread, document the:

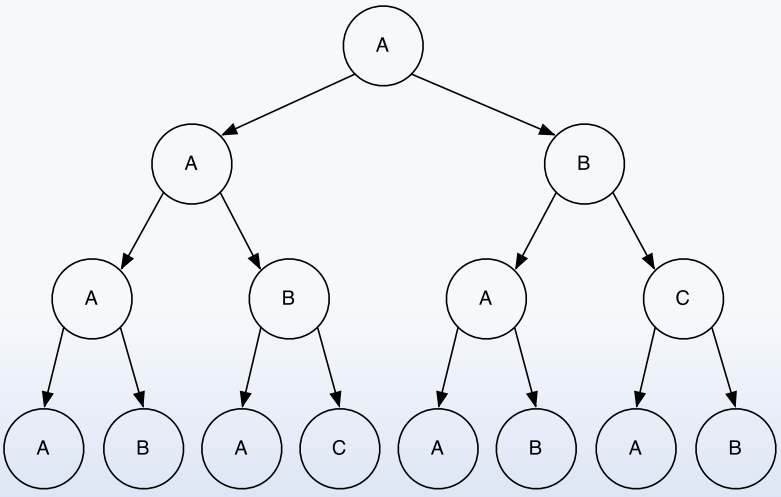
- States,
- Transitions, and
- Likelihood.

Threads should be ***four*** states long.



11

In Class Assignment: Identify Thread Chances



12

Risks in Software

- A risk is an unwanted event that has negative consequences (often due to incorrect estimation)
- Distinguish risks from other project events
  - **Risk impact**: the loss associated with the event
  - **Risk probability**: the likelihood that the event will occur
- Quantify the effect of risks
  - **Risk exposure** = (risk probability) x (risk impact)

13

Risk-Based Testing

Probability \ Impact	Negligible	Marginal	Critical	Catastrophic
Certain	High	High	Extreme	Extreme
Likely	Moderate	High	High	Extreme
Possible	Low	Moderate	High	Extreme
Unlikely	Low	Low	Moderate	Extreme
Rare	Low	Low	Moderate	High

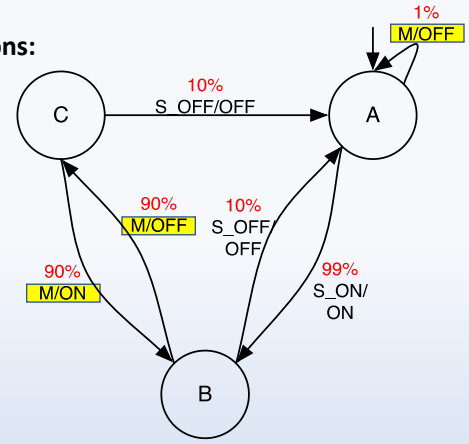
14

FSA Path Probabilities + Risks

Automatic Light Switch Finite State Machine

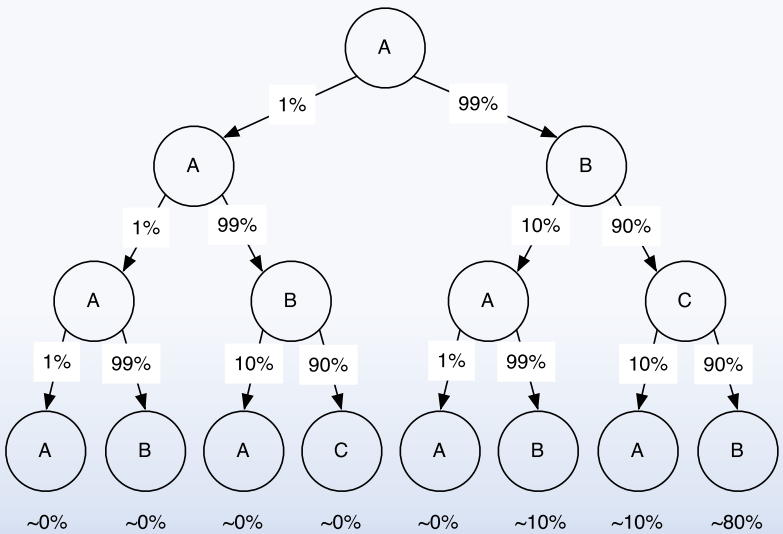
Risky (Motion-Based) Transitions:

- A to A,
- B to C,
- C to B



15

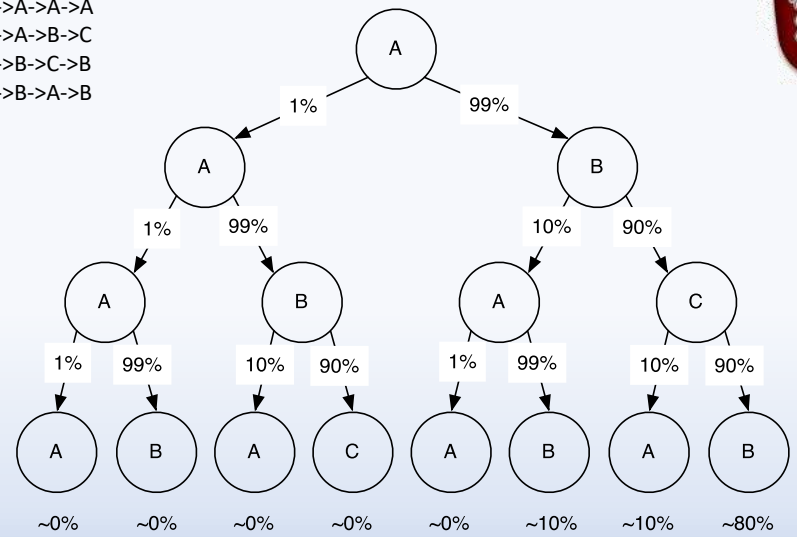
Given the likelihood, add the risk!



16

What path has the most risk probability?

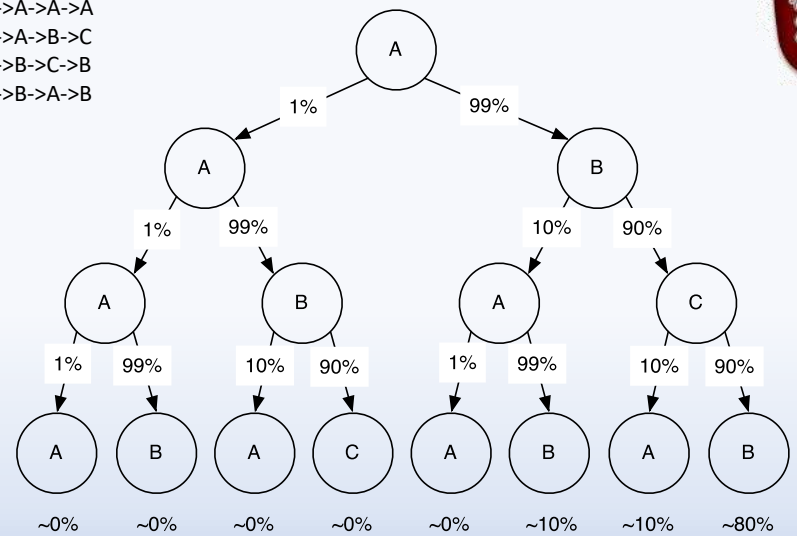
A. A->A->A->A
B. A->A->B->C
C. A->B->C->B
D. A->B->A->B



17

What path has the most risk exposure?

A. A->A->A->A
B. A->A->B->C
C. A->B->C->B
D. A->B->A->B



18

Discussion

- How does system testing differ from unit / module testing?
- How does system testing differ from integration testing?
- Could tests cover both unit, integration, and system testing?

19